

Practical 1

Name: Chetan Bapurao Jadhav

PRN: 202501040051

Division : CS1

Batch : C13

1.1.1 Calculate Momentum

Problem Statement:

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula: $p = m * v$

where:

m is the mass of the object (in kilograms).

v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Code:

```
mass = float(input())
velocity = float(input())
momentum= mass * velocity
print(f"{momentum:.2f} kgm/s")
```

Output:

Average time

0.044 s

44.50 ms

Maximum time

0.063 s

63.00 ms

✓ 2 out of 2 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1

63 ms

Debug

Expected output

Actual output

5.0

5.0

10.0

10.0

50.00kgm/s

50.00kgm/s

✓ Test case 2

59 ms

Debug

Expected output

Actual output

2.3

2.3

4.8

4.8

11.04kgm/s

11.04kgm/s

1.1.2 Conditional Calculation based on the Number of Digits

Problem Statement:

Write a Python program that accepts an integer as input. Depending on the number of digits in.

Constraints: $1 \leq n \leq 999$

Input Format:

- The input consists of a single integer.

Output Format:

- If is a single-digit number, print its square.
- If is a two-digit number, print its square root (rounded to two decimal places).
- If is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid"

Code:

```
import math
n=int(input())
sq = n*n
sq_root = math.sqrt(n)
cb_root = math.cbrt(n)
if n>1 and n<=9:
    print(f"{sq}")
elif n>=10 and n<=99:
    print(f"{sq_root:.2f}")
elif n>=100 and n<=999:
    print(f"{cb_root:.2f}")
else:
    print("Invalid")
```

Output:

Average time
0.020 s
20.43 ms

Maximum time
0.029 s
29.00 ms

✓ 4 out of 4 shown test case(s) passed

✓ 3 out of 3 hidden test case(s) passed

✓ Test case 1 14 ms Debug

Expected output

9

81

Actual output

9

81

✓ Test case 2 26 ms Debug

Expected output

166

5.50

Actual output

166

5.50

✓ Test case 3 15 ms Debug

Expected output

24

4.90

Actual output

24

4.90

✓ Test case 4 20 ms Debug

Expected output

3456

Invalid

Actual output

3456

Invalid

1.1.3 Days Between Two Dates

Problem Statement:

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format YYYY-MM-DD.
- The second line contains the second date in the format YYYY-MM-DD.

Output Format:

- An integer representing the number of days between the given dates.

Code:

```
from datetime import date
from datetime import datetime
date1 = input().strip()
date2 = input().strip()
d1 = datetime.strptime(date1, "%Y-%m-%d")
d2 = datetime.strptime(date2, "%Y-%m-%d")
difference = d2-d1
print(difference.days)
```

Output:

Average time
0.035 s
34.50 ms

Maximum time
0.064 s
64.00 ms

✓ 2 out of 2 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 64 ms Debug

Expected output	Actual output
2014-07-02	2014-07-02
2014-11-02	2014-11-02
123	123

✓ Test case 2 27 ms Debug

Expected output	Actual output
2014-07-02	2014-07-02
2014-07-11	2014-07-11
9	9

1.1.4 Reverse a Number

Problem Statement:

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Code:

```
n=int(input())
reverse=int(str(n)[::-1])
print(reverse)
```

Output:

Average time
0.017 s
16.60 ms

Maximum time
0.020 s
20.00 ms

✓ 2 out of 2 shown test case(s) passed

✓ 3 out of 3 hidden test case(s) passed

✓ Test case 1 18 ms Debug

Expected output

5367

7635

Actual output

5367

7635

✓ Test case 2 20 ms Debug

Expected output

0132

231

Actual output

0132

231

1.1.5 Multiplication Table

Problem Statement:

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:
<number> x <multiplier> = <result>

Code:

```
n = int(input())
for i in range(1,11):
    print(n, "x", i, "=", n*i)
```

Output:

Average time
0.021 s
20.75 ms

Maximum time
0.026 s
26.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1
26 ms
Debug

Expected output

Actual output

8

8

$8 \cdot x \cdot 1 = \cdot 8$

$8 \cdot x \cdot 1 = \cdot 8$

$8 \cdot x \cdot 2 = \cdot 16$

$8 \cdot x \cdot 2 = \cdot 16$

$8 \cdot x \cdot 3 = \cdot 24$

$8 \cdot x \cdot 3 = \cdot 24$

$8 \cdot x \cdot 4 = \cdot 32$

$8 \cdot x \cdot 4 = \cdot 32$

$8 \cdot x \cdot 5 = \cdot 40$

$8 \cdot x \cdot 5 = \cdot 40$

$8 \cdot x \cdot 6 = \cdot 48$

$8 \cdot x \cdot 6 = \cdot 48$

$8 \cdot x \cdot 7 = \cdot 56$

$8 \cdot x \cdot 7 = \cdot 56$

$8 \cdot x \cdot 8 = \cdot 64$

$8 \cdot x \cdot 8 = \cdot 64$

$8 \cdot x \cdot 9 = \cdot 72$

$8 \cdot x \cdot 9 = \cdot 72$

$8 \cdot x \cdot 10 = \cdot 80$

$8 \cdot x \cdot 10 = \cdot 80$

Test case 2
16 ms
Debug

Expected output

Actual output

5

5

$5 \cdot x \cdot 1 = \cdot 5$

$5 \cdot x \cdot 1 = \cdot 5$

$5 \cdot x \cdot 2 = \cdot 10$

$5 \cdot x \cdot 2 = \cdot 10$

$5 \cdot x \cdot 3 = \cdot 15$

$5 \cdot x \cdot 3 = \cdot 15$

$5 \cdot x \cdot 4 = \cdot 20$

$5 \cdot x \cdot 4 = \cdot 20$

$5 \cdot x \cdot 5 = \cdot 25$

$5 \cdot x \cdot 5 = \cdot 25$

$5 \cdot x \cdot 6 = \cdot 30$

$5 \cdot x \cdot 6 = \cdot 30$

$5 \cdot x \cdot 7 = \cdot 35$

$5 \cdot x \cdot 7 = \cdot 35$

$5 \cdot x \cdot 8 = \cdot 40$

$5 \cdot x \cdot 8 = \cdot 40$

$5 \cdot x \cdot 9 = \cdot 45$

$5 \cdot x \cdot 9 = \cdot 45$

$5 \cdot x \cdot 10 = \cdot 50$

$5 \cdot x \cdot 10 = \cdot 50$

1.2.1 Pass or Fail

Problem Statement:.

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer, the number of courses.
- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Code:

```
def calculate(marks, total_courses):
    if any(mark<40 for mark in marks):
        return "Fail"
    total_marks = sum(marks)
    aggregate_percentage = ((total_marks)/(total_courses*100))*100
    if aggregate_percentage > 75:
        grade = "Distinction"
    elif aggregate_percentage >= 60:
        grade = "First Division"
    elif aggregate_percentage >= 50:
        grade = "Second Division"
    elif aggregate_percentage >= 40:
        grade = "Third Division"
    return (aggregate_percentage, grade)
num_courses = int(input())
marks = list(map(int,input().split()))
result = calculate(marks, num_courses)
if result == 'Fail':
    print("Fail")
else:
    aggregate_percentage, grade = result
    print("Aggregate Percentage:",f"{aggregate_percentage:.2f}")
    print(f"Grade: {grade}")
```

Output:

✓ Test case 1 42 ms Debug Expected output 4 55 65 78 89 Aggregate Percentage: 71.75 Grade: First Division	Actual output 4 55 65 78 89 Aggregate Percentage: 71.75 Grade: First Division
✓ Test case 2 38 ms Debug Expected output 5 56 78 97 86 93 Aggregate Percentage: 82.00 Grade: Distinction	Actual output 5 56 78 97 86 93 Aggregate Percentage: 82.00 Grade: Distinction
✓ Test case 3 35 ms Debug Expected output 5 99 85 43 97 34 Fail	Actual output 5 99 85 43 97 34 Fail
✓ Test case 4 30 ms Debug Expected output 3 55 54 53 Aggregate Percentage: 54.00 Grade: Second Division	Actual output 3 55 54 53 Aggregate Percentage: 54.00 Grade: Second Division
✓ Test case 5 23 ms Debug Expected output 5 40 45 42 48 41 Aggregate Percentage: 43.20 Grade: Third Division	Actual output 5 40 45 42 48 41 Aggregate Percentage: 43.20 Grade: Third Division

1.2.2 Fibonacci Series

Problem Statement:

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

- A single integer representing the number of terms to generate.

Output Format:

- A single line containing the first terms of the Fibonacci sequence, separated by spaces.

Code:

```
def fibonacci(n, a=0, b=1):
    if n==0:
        return a
    else:
        return fibonacci(n-1,b,a+b)

n=int(input())
for i in range(n):
    print(fibonacci(i),end=" ")
```

Output:

Average time
0.015 s
15.25 ms

Maximum time
0.021 s
21.00 ms

✓ 2 out of 2 shown test case(s) passed
✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 12 ms Debug

Expected output
5
0 1 1 2 3

Actual output
5
0 1 1 2 3

✓ Test case 2 12 ms Debug

Expected output
15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

Actual output
15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

1.2.3 Pattern-1

Problem Statement:

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Code:

```
rows = int(input())
for i in range(1, rows+1):
    for j in range(i):
        print("* ",end = "")
    print()
```

Output:

Average time
0.027 s
27.17 ms

Maximum time
0.036 s
36.00 ms

✓ 2 out of 2 shown test case(s) passed
✓ 4 out of 4 hidden test case(s) passed

✓ Test case 1 29 ms Debug

Expected output

Actual output

5

* , ⌵

* , * , ⌵

* , * , * , ⌵

* , * , * , * , ⌵

* , * , * , * , * , ⌵

* , * , * , * , * , * , ⌵

5

* , ⌵

* , * , ⌵

* , * , * , ⌵

* , * , * , * , ⌵

* , * , * , * , * , ⌵

✓ Test case 2 36 ms Debug

Expected output

Actual output

4

* , ⌵

* , * , ⌵

* , * , * , ⌵

* , * , * , * , ⌵

* , * , * , * , * , ⌵

1.2.4 Pattern-2

Problem Statement:

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Code:

```
n=int(input())
for i in range(1, n+1):
    for j in range(1, i+1):
        print(j,end=" ")
    print()
```

Output:

Average time

0.018 s

17.75 ms

Maximum time

0.021 s

21.00 ms

✓ 2 out of 2 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1

13 ms

Debug

Expected output

Actual output

5	5
1	1
1 2	1 2
1 2 3	1 2 3
1 2 3 4	1 2 3 4
1 2 3 4 5	1 2 3 4 5

✓ Test case 2

21 ms

Debug

Expected output

Actual output

8	8
1	1
1 2	1 2
1 2 3	1 2 3
1 2 3 4	1 2 3 4
1 2 3 4 5	1 2 3 4 5
1 2 3 4 5 6	1 2 3 4 5 6
1 2 3 4 5 6 7	1 2 3 4 5 6 7
1 2 3 4 5 6 7 8	1 2 3 4 5 6 7 8